Assignment IX – Group Project: Front End

RTL to Gate-Level Netlist of a Streaming Harris Corner Detection Accelerator (harris_top)

BEVD301P ASIC Design Lab · Fall Semester 2026–27 · Due 31 October 2026

Team Details

Name	Registration Number	Lab Slot
Saket Sodani	24BVD0069	L9 + L10
Advay Gopalakrishnan	24BVD0191	L21 + L22
Swajit Jain	24BVD0022	L21 + L22
Krithik Sai	24BVD0039	L21 + L22
Arrnav Trisal	24BVD0064	L21 + L22
Ashmit Singh	24BVD0027	L21 + L22

1. Introduction

This report documents the front-end stages of our group project, **harris_top**, a streaming Harris corner detection accelerator written in synthesizable SystemVerilog. The design accepts a raster-ordered stream of 8-bit grayscale pixels, one per clock, and for every pixel whose full neighbourhood is available it outputs a binary corner decision together with the Harris response. The report covers the RTL implementation, functional verification in Cadence Xcelium, and logic synthesis in Cadence Genus. The synthesized netlist was then checked for logical equivalence with Conformal LEC and for timing with Tempus before being handed to the back end (Assignment X).

Table 1: *Front-end flow and tools used*

Stage	Tool	Inputs	Outputs
RTL design	SystemVerilog	Specification	7 RTL modules (rtl/*.sv)
Functional simulation	Cadence Xcelium (xrun 24.09)	RTL, self-checking testbench	PASS/FAIL log
Logic synthesis	Cadence Genus 21.14	RTL, slow.lib, constraints	Netlist, SDC, SDF, reports
Equivalence check	Cadence Conformal LEC	RTL (golden), netlist (revised)	Verification report
Pre-layout STA	Cadence Tempus 22.11	Netlist, slow.lib, SDC	Setup and hold reports

2. RTL Implementation

2.1 Architecture Overview

The detector is a linear pipeline of six modules (Figure 1). Pixels enter in raster order with a valid strobe; the design stores only a few image rows in line buffers and never a full frame. Every stage registers its result, and a valid bit travels alongside the data so that each output is aligned with its decision. A low pixel_valid acts as a stall: counters, line buffers and pipeline registers hold their state.

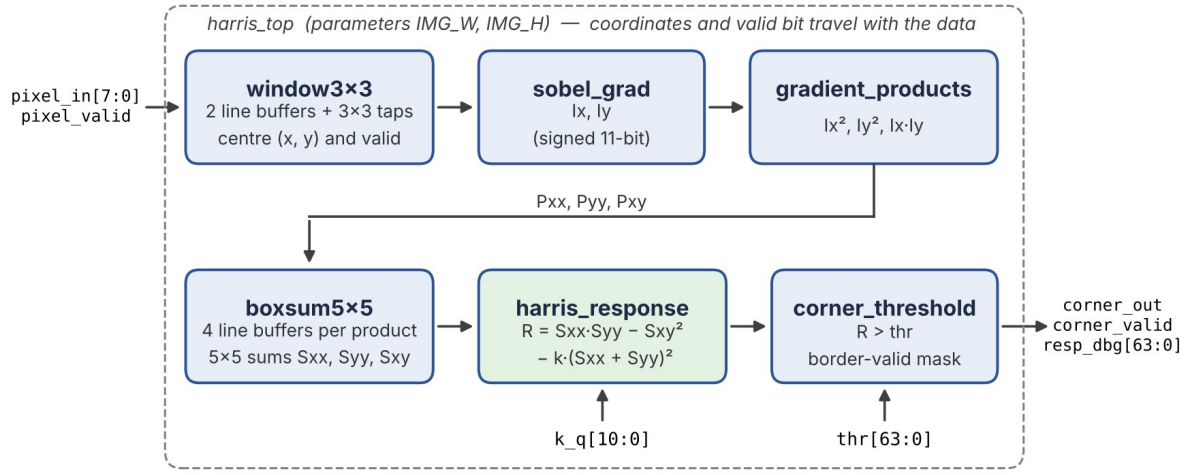


Figure 1: *Block diagram of harris_top. The image size is set by the parameters IMG_W and IMG_H (64 × 64 for synthesis).*

2.2 Design Hierarchy

Table 2: *RTL module hierarchy (instance names from the synthesized netlist)*

Instance	Module	Function and implementation
—	harris_top	Top level. Parameters IMG_W, IMG_H (image size, 64 × 64 for synthesis), K_FRAC = 10 (fractional bits of k) and R_W = 54 + K_FRAC = 64 (response width).
u_win	window3x3	Column and row counters, two line buffers holding the previous two rows and three horizontal taps per row, giving a 3 × 3 window and a window-valid flag.
u_sobel	sobel_grad	3 × 3 Sobel kernels using shifts, additions and subtractions only (no multipliers).
u_prod	gradient_products	Ix^2, Iy^2 and Ix·Iy with three 11 × 11 multipliers. The fourth term is not computed because the matrix M is symmetric.
u_box	boxsum5x5	5 × 5 window sums, built from three instances of a reusable channel module boxsum_ch (u_xx, u_yy for the 20-bit unsigned products, u_xy for the 21-bit signed product). Each channel keeps four line buffers and a five-entry horizontal shift register. It works on the 62 × 62 gradient image left after the Sobel border.

Instance	Module	Function and implementation
u_resp	harris_response	Scaled Harris response with the run-time constant k_q (Section 2.4); pipelined internally, e.g. the Sxy^2 product is registered (p_sxy2_reg).
u_thr	corner_threshold	Compares the response with thr, asserts corner_out and applies the border-valid mask.

2.3 Top-Level Interface

```
[24bvd0039@cadence-server rtl]$ cat harris_top.sv

module harris_top #(
  parameter int IMG_W  = 64,     // >= 7 to produce any output
  parameter int IMG_H  = 64,     // >= 7
  parameter int K_FRAC = 10,
  parameter int R_W    = 54 + K_FRAC
) (
  input  logic                          clk,
  input  logic                          rst_n,        // synchronous, active low
  input  logic [7:0]                    pixel_in,
  input  logic                          pixel_valid,
  input  logic [K_FRAC:0]               k_q,          // k = k_q/2^K_FRAC ; typical 41 (~0.04)
  input  logic signed [R_W-1:0]         thr,          // threshold in scaled units (R * 2^K_FRAC)
  output logic                          corner_out,
  output logic                          corner_valid,
  output logic signed [R_W-1:0]         resp_dbg      // scaled response aligned with corner_out
);
```

Figure 2: *harris_top.sv: parameters and port list*

Table 3: *Top-level ports of harris_top*

Port	Dir.	Width	Function
clk	in	1	System clock
rst_n	in	1	Synchronous, active-low reset
pixel_in	in	8	Unsigned 8-bit grayscale pixel
pixel_valid	in	1	Pixel strobe; low = stall the whole pipeline
k_q	in	11 (K_FRAC + 1)	Harris constant k = k_q / 2^K_FRAC = k_q / 1024; typical k_q = 41 (k ≈ 0.04)
thr	in	64 (R_W)	Signed threshold in scaled units (R × 2^K_FRAC)
corner_out	out	1	1 when the scaled response is greater than thr
corner_valid	out	1	Output corresponds to a complete, legal window
resp_dbg	out	64 (R_W)	Signed scaled response, aligned with corner_out (for debug and verification)

2.4 Datapath

Sobel gradients. For the 3 × 3 window p00…p22 (row, column), the gradients use shifts and adds only:

$$Ix = (p02 + 2{\cdot}p12 + p22) - (p00 + 2{\cdot}p10 + p20)$$
$$Iy = (p20 + 2{\cdot}p21 + p22) - (p00 + 2{\cdot}p01 + p02)$$

For 8-bit pixels $|Ix|, |Iy| \le 1020$, which fits a signed 11-bit word.

Gradient products and integration. The three distinct products $Pxx = Ix^2$, $Pyy = Iy^2$ and $Pxy = Ix \cdot Iy$ are summed over a 5 × 5 window centred on the pixel:

$$Sab(x, y) = \Sigma_{i=-2..2} \Sigma_{j=-2..2} Pab(x + i, y + j), \quad ab \in \{xx, yy, xy\}$$

Harris response and decision. With k = k_q / 2^K_FRAC, the response is computed scaled by 2^K_FRAC so that no fractional bits are ever discarded:

$$R_s = 2\text{^}K_FRAC \cdot (Sxx \cdot Syy - Sxy^2) - k_q \cdot (Sxx + Syy)^2 \;=\; 2\text{^}K_FRAC \cdot R$$

$$corner_out = (R_s > thr)$$

The threshold is given in the same scaled units, so the comparison is exact. Every width is derived from the worst-case value for 8-bit input:

Table 4: *Arithmetic widths from worst-case bounds (8-bit input)*

Quantity	Format	Worst-case range
Ix, Iy	signed 11-bit	±1,020
Pxx, Pyy	unsigned 20-bit	0 … 1,040,400
Pxy	signed 21-bit	±1,040,400
Sxx, Syy	unsigned 25-bit	0 … 26,010,000
Sxy	signed 26-bit	±26,010,000
Sxx·Syy	unsigned 50-bit	$< 6.8 \times 10^{14}$
R_s (resp_dbg)	signed 64-bit (R_W = 54 + K_FRAC)	Exact; no truncation

2.5 Key Design Decisions

1. **Streaming with line buffers, no frame store.** One pixel is accepted per clock. Only two pixel rows (window3x3) and four rows of each product (boxsum5x5) are stored, so memory grows with image width, not image area.
2. **Exploiting the symmetry of M.** Only three of the four structure-tensor entries are computed, saving a multiplier and a whole channel of line buffers.
3. **5 × 5 box window instead of a Gaussian.** A uniform window needs no multipliers for the weights; the common scale factor is absorbed into the threshold.
4. **One reusable box-sum channel.** boxsum5x5 instantiates the same parameterised boxsum_ch three times with different input width and signedness, instead of three hand-written copies.
5. **Exact, scaled integer arithmetic.** The response is kept in units of 2^K_FRAC, so the k multiplication loses no bits and the threshold comparison is exact. This makes results deterministic and easy to check.
6. **Run-time k and threshold.** k_q and thr are inputs rather than compile-time constants, so the same netlist can be tuned without re-synthesis; the testbench

uses this to sweep k_q and thr (Section 3). resp_dbg exposes the response for debugging.

7. **Valid, stall and reset handling.** pixel_valid low freezes the whole pipeline; rst_n is synchronous and active low.
8. **Border masking.** corner_valid is asserted only where both the 3 × 3 Sobel window and the 5 × 5 integration window are complete, i.e. $x \in [3, \text{IMG_W} - 4]$ and $y \in [3, \text{IMG_H} - 4]$. A 13 × 11 image therefore gives 7 × 5 = 35 outputs, which the testbench checks.
9. **Parameterised image size.** IMG_W and IMG_H set the line-buffer depth; 64 × 64 was used for synthesis and a 13 × 11 image for simulation to keep run times short.

3. Functional Verification

3.1 Testbench and Approach

The self-checking testbench tb_harris_top (tb_harris.sv) was run in Cadence Xcelium with a 13 × 11 image, a pipeline latency of 10 cycles and random seed 1. The testbench is self-contained: the test images, configurations and expected results all come from the testbench files themselves, with no external vector files. It checks every output against the expected result and reports PASS or FAIL for each test.

The 28 tests fall into six groups:

Table 5: *Test plan*

Group	Tests	What it exercises
Edges (no corners)	Flat image, horizontal, vertical and diagonal edge	Edges and flat regions must give R ≤ thr, so no corners
Corner patterns	Single bright pixel, bright square (4 corners), checkerboard 1 px and 2 px, ramp, random	Corners detected where expected
Stalls (valid gaps)	Random with 35 % and 70 % gaps, square with 50 % gaps, 200-cycle idle mid-frame	pixel_valid low must freeze the pipeline without corrupting results
Threshold and k_q limits	thr = an actual R (strict >), thr = max R, thr = most negative, k_q = 0, k_q = 2047 (max), 0/255 noise	Strict comparison, extreme thresholds, extreme k and the largest possible sums (overflow check)
Frames and reset	3 different frames back-to-back, reset mid-frame then a clean frame	Frame wrap-around and recovery from reset
Randomised configurations	6 configurations with random pattern, gap rate (15–40 %), k_q and thr; 2 frames each	Combinations not covered by the directed tests

3.2 Simulation Results

All 28 tests passed: **1,260 outputs checked, 672 corners detected, 0 mismatches**. Each single-frame test checks 35 outputs (the 7 × 5 interior of the 13 × 11 image); the back-to-back test checks 105 and each randomised configuration 70 (two frames).

Table 6: *Regression summary (from the Xcelium log)*

Group	Tests	Outputs checked	Corners	Result
Edges (no corners)	4	140	0	PASS
Corner patterns	6	210	163	PASS
Stalls (valid gaps)	4	140	140	PASS
Threshold and k_q limits	6	210	124	PASS
Frames and reset	2	140	105	PASS
Randomised configurations	6	420	140	PASS
Total	**28**	**1,260**	**672**	**PASS**

The edge cases behave as specified: a flat image gives R = 0, which is not greater than thr = 0, so no corner; k_q = 0 makes R equal to the scaled determinant; thr equal to the maximum R gives no corners because the comparison is strict; and the 0/255 noise test drives the sums to their extremes without overflow.

```
xcelium> run
=== tb_harris_top: IMG 13x11, LAT=10, 35 interior outputs/frame, seed=1 ===
PASS  flat image (R=0, thr=0 -> no corner) outputs=35 corners=0
PASS  horizontal edge                     outputs=35 corners=0
PASS  vertical edge                       outputs=35 corners=0
PASS  diagonal edge                       outputs=35 corners=0
PASS  single bright pixel                 outputs=35 corners=35
PASS  bright square (4 corners)           outputs=35 corners=35
PASS  checkerboard 1px                    outputs=35 corners=0
PASS  checkerboard 2px                    outputs=35 corners=35
PASS  ramp                                outputs=35 corners=23
PASS  random                              outputs=35 corners=35
PASS  random, 35% gaps                    outputs=35 corners=35
PASS  random, 70% gaps                    outputs=35 corners=35
PASS  square, 50% gaps                    outputs=35 corners=35
PASS  thr = an actual R (strict >)        outputs=35 corners=21
PASS  thr = max R (expect no corners)     outputs=35 corners=0
PASS  thr = most negative (all corners)   outputs=35 corners=35
PASS  k_q = 0 (R = det*2^10)              outputs=35 corners=35
PASS  k_q = max (2047)                    outputs=35 corners=0
PASS  0/255 noise (extreme sums)          outputs=35 corners=33
PASS  3 different frames back-to-back     outputs=105 corners=70
PASS  long idle (200 clk) mid-frame       outputs=35 corners=35
PASS  reset mid-frame, then clean frame   outputs=35 corners=35
PASS  random cfg 0 (pat=1 gap=15% k_q=1427 thr=2) outputs=70 corners=0
PASS  random cfg 1 (pat=0 gap=52% k_q=1665 thr=1) outputs=70 corners=0
PASS  random cfg 2 (pat=6 gap=40% k_q=747 thr=3) outputs=70 corners=70
PASS  random cfg 3 (pat=4 gap=26% k_q=118 thr=2) outputs=70 corners=0
PASS  random cfg 4 (pat=1 gap=39% k_q=517 thr=3) outputs=70 corners=70
PASS  random cfg 5 (pat=5 gap=21% k_q=1402 thr=0) outputs=70 corners=0
--------------------------------------------------------------------------
TEST PASSED tb_harris_top 13x11: 28 tests, 1260 outputs checked, 672 corners
Simulation complete via $finish(1) at time 77130 NS + 0
./tb_harris.sv:387     $finish;
xcelium> exit
TOOL:   xrun(64)        24.09-s005: Exiting on Oct 30, 2026 at 15:46:54 IST  (total: 00:00:02)
[24bvd0039@cadence-server tb]$
```

Figure 3: *Xcelium regression log: all 28 tests pass (1,260 outputs, 672 corners)*

4. Logic Synthesis

4.1 Synthesis Flow

Synthesis was run with Cadence Genus 21.14 using the slow-corner Liberty file of the 90 nm generic standard-cell library (gsclib090), the same library whose LEF is used in Innovus. The script (genus.tcl) takes the corner, clock period and image size from environment variables, with defaults CORNER = slow, CLK_PERIOD = 2.25 ns and IMG_W = IMG_H = 64. The main steps are:

1. **Read library and RTL.** read_libs ../synthesis/slow.lib, then read_hdl -sv on the seven RTL files in bottom-up order.
2. **Keep hierarchy.** set_db auto_ungroup none keeps the module instances (u_win, u_box, u_resp, ...), so area is reported per block and the instances remain visible in Innovus.
3. **Elaborate with parameters.** elaborate harris_top -parameters with IMG_W = IMG_H = 64. Genus renames a top level elaborated with non-default parameters, so the script reads back the elaborated name (harris_top_IMG_W64_IMG_H64) instead of hard-coding it; the same name is used in LEC, Tempus and Innovus.
4. **Design checks.** check_design -unresolved and check_design -all before synthesis.
5. **Constraints.** Clock, input/output delays and design-rule limits (Section 4.2).
6. **Synthesis at high effort.** syn_generic, syn_map and syn_opt, each at high effort, writing the netlist after each step.
7. **Outputs.** Final netlist (harris_top_netlist_slow.v), SDC (harris_top_slow.sdc) for the back end, SDF for gate-level simulation, and area (flat and per-instance), gates, power, timing and QoR reports.

```
read_libs ../synthesis/${corner}.lib
set_db auto_ungroup none
read_hdl -sv $rtl_files
elaborate $design -parameters [list [list IMG_W $img_w] [list IMG_H $img_h]]
create_clock -name clk -period $clk_period [get_ports clk]
set_input_delay  0.06 $in_ports     -clock [get_clocks clk]
set_output_delay 0.04 [all_outputs] -clock [get_clocks clk]
set_db syn_generic_effort high ; set_db syn_map_effort high ; set_db syn_opt_effort high
syn_generic ; syn_map ; syn_opt
```

4.2 Timing Constraints

The design was first synthesized with an aggressive 2.25 ns clock and tight design-rule limits. That run missed timing by 1.465 ns (Section 4.3), so the clock was relaxed by editing the SDC written by Genus; the same edit relaxed the transition and fanout limits. The edited SDC is the one used for STA and for the whole back-end flow.

Table 7: *Synthesis and final timing constraints*

Constraint	Synthesis run (genus.tcl)	Final SDC (Innovus)
Clock period	2.25 ns (444 MHz)	4.0 ns (250 MHz)

Constraint	Synthesis run (genus.tcl)	Final SDC (Innovus)
Input delay (all inputs except clk)	0.06 ns	0.06 ns
Output delay (all outputs)	0.04 ns	0.04 ns
Max transition	0.15 ns	0.5 ns
Max fanout	4	32

Why the new values: the 2.25 ns critical path needs about 3.57 ns of logic plus 0.15 ns setup, so any period below about 3.72 ns fails; 4 ns gives margin for clock skew and wire delay after layout. A max-fanout of 4 and a max-transition of 0.15 ns are very strict for a 90 nm slow corner: nets such as the reset, the pipeline enable and the 64-bit threshold bus each drive many flip-flops, and meeting those limits would need deep buffer trees that cost area and delay. The final limits of 0.5 ns and 32 keep the checks meaningful without forcing that buffering.

4.3 Synthesis Results

Area

Table 8: *Area by instance (harris_top_area_report.rpt, slow corner)*

Instance	Module	Cells	Cell area (μm²)	Share
harris_top_IMG_W64_IMG_H64	(top)	71,778	593,021.31	100 %
u_box	boxsum5x5	50,033	455,095.21	76.74 %
u_xy	boxsum_ch (21-bit signed)	17,845	161,765.43	27.28 %
u_yy	boxsum_ch (20-bit)	16,058	146,469.23	24.70 %
u_xx	boxsum_ch (20-bit)	16,015	146,096.08	24.64 %
u_resp	harris_response	17,142	93,409.03	15.75 %
u_win	window3x3	2,645	31,300.09	5.28 %
u_prod	gradient_products	1,352	8,172.25	1.38 %
u_sobel	sobel_grad	299	2,699.11	0.46 %
u_thr	corner_threshold	303	2,328.98	0.39 %

The total cell area is **593,021 μm² (≈ 0.59 mm²)** with 71,778 cells, of which **16,741 are flip-flops** (sequential instances) and 55,037 combinational. Storage dominates: boxsum5x5 alone is 76.7 % of the area. Its twelve line buffers (four per channel, 62 entries of 20 or 21 bits) plus the two 64 × 8-bit rows in window3x3 come to about $4 \cdot 62 \cdot (20 + 20 + 21) + 2 \cdot 64 \cdot 8 \approx 16{,}150$ bits, i.e. roughly 96 % of all flip-flops. Because the course flow has no SRAM macros, every one of these bits is a flip-flop. The arithmetic-heavy harris_response is second at 15.8 %.

```
[24bvd0069@cadence-server reports_slow]$ cat harris_top_area_report.rpt
====================================================
  Generated by:           Genus(TM) Synthesis Solution 21.14-s082_1
  Generated on:           Oct 26 2026  04:38:44 pm
  Module:                 harris_top_IMG_W64_IMG_H64
  Technology library:     slow
  Operating conditions:   slow (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
====================================================

        Instance                          Module                 Cell Count  Cell Area  Net Area   Total Area   Wireload
-------------------------------------------------------------------------------------------------------------------------
harris_top_IMG_W64_IMG_H64                                           71778 593021.310     0.000   593021.310 <none> (D)
  u_box                   boxsum5x5_IMG_W62_IMG_H62                  50033 455095.208     0.000   455095.208 <none> (D)
    u_xx                  boxsum_ch_IN_W20_SGN0_IMG_W62_CWID6        16015 146096.081     0.000   146096.081 <none> (D)
    u_xy                  boxsum_ch_IN_W21_SGN1_IMG_W62_CWID6        17845 161765.425     0.000   161765.425 <none> (D)
    u_yy                  boxsum_ch_IN_W20_SGN0_IMG_W62_CWID6_673    16058 146469.233     0.000   146469.233 <none> (D)
  u_prod                  gradient_products                           1352   8172.249     0.000     8172.249 <none> (D)
  u_resp                  harris_response_K_FRAC10_R_W64             17142  93409.029     0.000    93409.029 <none> (D)
  u_sobel                 sobel_grad                                   299   2699.105     0.000     2699.105 <none> (D)
  u_thr                   corner_threshold_R_W64                       303   2328.981     0.000     2328.981 <none> (D)
  u_win                   window3x3_IMG_W64_IMG_H64                   2645  31300.086     0.000    31300.086 <none> (D)
  (D) = wireload is default in technology library
[24bvd0069@cadence-server reports_slow]$
```

Figure 4: *Genus area report (per instance)*

The gates report (Table 9) splits the same area by cell type. Flip-flops take 58.6 % of the area with only 23 % of the cells, because each flip-flop is far larger than a simple gate. For the multipliers and adders Genus uses dedicated arithmetic cells: 510 full-adder cells (ADDFX/ADDFHX variants), half adders (ADDHX1) and three-input XOR/XNOR cells (XOR3, XNOR3).

Table 9: *Area by cell type (harris_top_gates_report.rpt)*

Cell type	Instances	Area (μm²)	Area %
Sequential	16,741	347,310.38	58.6 %
Logic	30,101	177,209.97	29.9 %
Inverter	23,211	59,282.68	10.0 %
Buffer	1,725	9,218.29	1.6 %
Physical cells	0	0.00	0.0 %
Total	**71,778**	**593,021.31**	**100 %**

```
XNOR2X1          279    2322.926    slow
XNOR2X2           13     137.756    slow
XNOR2X4            2      22.707    slow
XNOR2XL          148    1232.233    slow
XNOR3X1           94    2134.458    slow
XNOR3XL            7     158.949    slow
XOR2X1             1       8.326    slow
XOR2XL           127    1057.389    slow
XOR3X1           137    3110.859    slow
XOR3XL             9     204.363    slow
-----------------------------------------
total          71778  593021.310


     Type      Instances    Area    Area %
-----------------------------------------
sequential         16741 347310.377   58.6
inverter           23211  59282.679   10.0
buffer              1725   9218.285    1.6
logic              30101 177209.969   29.9
physical_cells         0      0.000    0.0
-----------------------------------------
total              71778 593021.310  100.0
```

Figure 5: *Genus gates report: end of the cell list and the summary by cell type*

Timing (at the 2.25 ns synthesis clock)

Table 10: *Genus timing at 2.25 ns (timing and QoR reports)*

Metric	Value
Clock period	2.25 ns
Worst slack (WNS)	−1.465 ns (VIOLATED)
Total negative slack (TNS)	−254.62 ns
Violating paths	378
Start point	u_box/u_xy/sum_out_reg[9] (Sxy box-sum output register)
End point	u_resp/p_sxy2_reg[42] (Sxy^2 product register)
Data path delay	3.569 ns
Setup time / required time	0.146 ns / 2.104 ns
Clock fanout (ideal clock)	16,741 flip-flops

The critical path is the 26 × 26-bit Sxy^2 multiplier in harris_response: it runs from the Sxy register in boxsum5x5 through more than 30 gate levels to the p_sxy2 register, taking 3.57 ns on its own. Since it is already one register-to-register stage, the 2.25 ns target could only be met by splitting the multiplier across two pipeline stages; we chose instead to relax the clock to 4 ns.

```
[24bvd0069@cadence-server reports_slow]$ cat harris_top_timing_report.rpt
====================================================
  Generated by:           Genus(TM) Synthesis Solution 21.14-s082_1
  Generated on:           Oct 26 2026  04:38:50 pm
  Module:                 harris_top_IMG_W64_IMG_H64
  Operating conditions:   slow (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
====================================================


Path 1: VIOLATED (-1465 ps) Setup Check with Pin u_resp/p_sxy2_reg[42]/CK->D
          Group: clk
     Startpoint: (R) u_box/u_xy/sum_out_reg[9]/CK
          Clock: (R) clk
       Endpoint: (R) u_resp/p_sxy2_reg[42]/D
          Clock: (R) clk

                    Capture       Launch
       Clock Edge:+    2250            0
      Src Latency:+       0            0
      Net Latency:+       0 (I)        0 (I)
          Arrival:=    2250            0

            Setup:-     146
    Required Time:=    2104
     Launch Clock:-       0
        Data Path:-    3569
            Slack:=   -1465

#-------------------------------------------------------------------------------------------------------
#        Timing Point          Flags    Arc    Edge   Cell     Fanout Load Trans Delay Arrival Instance
#                                                                (fF)  (ps)  (ps)   (ps)  Location
#-------------------------------------------------------------------------------------------------------
  u_box/u_xy/sum_out_reg[9]/CK -       -      R     (arrival)  16741    -     0     0       0    (-,-)
  u_box/u_xy/sum_out_reg[9]/Q  -       CK->Q  F     SDFFQX4        5 19.6    62   350     350    (-,-)
  u_resp/g7025/Y               -       A->Y   R     NOR2X2         1  5.5    72    80     430    (-,-)
  u_resp/g6999/Y               -       A->Y   F     NAND2X2        1 10.9    95    92     522    (-,-)
  u_resp/g6971/Y               -       B->Y   R     NOR2X4         3 13.2    84    87     609    (-,-)
  u_resp/g46565/Y              -       A->Y   F     INVX1          1 11.1    89    87     696    (-,-)
```

Figure 6: *Genus timing report at 2.25 ns: worst path summary*

```
#-------------------------------------------------------------------------------------------------------
#        Timing Point          Flags    Arc    Edge   Cell     Fanout Load Trans Delay Arrival Instance
#                                                                (fF)  (ps)  (ps)   (ps)  Location
#-------------------------------------------------------------------------------------------------------
  u_box/u_xy/sum_out_reg[9]/CK -       -      R     (arrival)  16741    -     0     0       0    (-,-)
  u_box/u_xy/sum_out_reg[9]/Q  -       CK->Q  F     SDFFQX4        5 19.6    62   350     350    (-,-)
  u_resp/g7025/Y               -       A->Y   R     NOR2X2         1  5.5    72    80     430    (-,-)
  u_resp/g6999/Y               -       A->Y   F     NAND2X2        1 10.9    95    92     522    (-,-)
  u_resp/g6971/Y               -       B->Y   R     NOR2X4         3 13.2    84    87     609    (-,-)
  u_resp/g46565/Y              -       A->Y   F     INVX1          1 11.1    89    87     696    (-,-)
  u_resp/g66677/Y              -       A->Y   R     NOR2X4         4 13.4    85    93     788    (-,-)
  u_resp/g64887/Y              -       A->Y   F     CLKINVX2       2 10.6    55    58     846    (-,-)
  u_resp/g64795/Y              -       B->Y   R     NOR2X2         1  2.8    52    55     901    (-,-)
  u_resp/g43243/Y              -       A1N->Y R     AOI2BB1X4      4 13.3    90   154    1054    (-,-)
  u_resp/g43242/Y              -       A->Y   F     CLKINVX2       2 10.4    56    58    1112    (-,-)
  u_resp/fopt48919/Y           -       A->Y   R     CLKINVX3       2 15.8    54    59    1172    (-,-)
  u_resp/g70526/Y              -       B1->Y  F     AOI22X2        2  6.8   110    73    1245    (-,-)
  u_resp/g65157/Y              -       A->Y   R     NOR2XL         2  3.3   126   127    1372    (-,-)
  u_resp/g78247/Y              -       AN->Y  R     NAND2BX2       2  6.4    63   152    1524    (-,-)
  u_resp/g75844/Y              -       C->Y   F     XNOR3X1        4  8.6   136   210    1734    (-,-)
  u_resp/g48308/Y              -       B->Y   R     NAND2BXL       1  1.7    65    72    1806    (-,-)
  u_resp/g48296/Y              -       B0->Y  F     OAI21XL        1  3.6   129   108    1914    (-,-)
  u_resp/g78110/Y              -       B->Y   R     CLKXOR2X1      3  7.9   100   218    2133    (-,-)
  u_resp/fopt66853/Y           -       A->Y   F     CLKINVX1       1  2.7    46    45    2178    (-,-)
  u_resp/g65067/Y              -       B->Y   R     MXI2X1         2  4.2   113   108    2286    (-,-)
  u_resp/g66394/Y              -       A->Y   F     XOR3X1         2  8.0   132   415    2701    (-,-)
  u_resp/g66395/Y              -       B->Y   R     NAND2X2        2  8.0    68    76    2777    (-,-)
  u_resp/g65598/Y              -       B->Y   F     NAND2X2        2 11.1    96    89    2866    (-,-)
  u_resp/g66698/Y              -       A->Y   R     NOR2X2         1  3.8    68    77    2943    (-,-)
  u_resp/g66697/Y              -       B->Y   R     CLKAND2X3      1  5.3    39   102    3045    (-,-)
  u_resp/g66691/Y              -       B1->Y  F     AOI22X2        1  7.6   116    73    3118    (-,-)
  u_resp/g78604/Y              -       B->Y   F     CLKAND2X6      9 33.8    66   158    3276    (-,-)
  u_resp/g74151/Y              -       A->Y   R     INVX2          4  8.8    49    57    3333    (-,-)
  u_resp/g51647/Y              -       B->Y   F     NAND2X1        2  3.4    68    66    3400    (-,-)
  u_resp/g45822/Y              -       A1N->Y F     OAI2BB1X1      1  1.7    54   117    3517    (-,-)
  u_resp/g47282/Y              -       B->Y   R     NAND2XL        1  1.7    67    52    3569    (-,-)
  u_resp/p_sxy2_reg[42]/D      <<<     -      R     DFFRX2         1    -     -     0    3569    (-,-)
#-------------------------------------------------------------------------------------------------------

[24bvd0069@cadence-server reports_slow]$
```

Figure 7: *Genus timing report: full critical path through u_resp*

The QoR report shows the failure was not limited to one path: 378 paths violate at 2.25 ns with a total negative slack of −254.6 ns, which confirms the clock target itself, not a

single path, had to change. It also lists the run statistics: 9 hierarchical instances, a maximum fanout of 16,741 on clk (one load per flip-flop, before clock-tree synthesis), an average fanout of 2.5, and a run time of about 72 minutes (4,328 s CPU) with a peak of 1.79 GB memory.

```
[24bvd0069@cadence-server reports_slow]$ cat harris_top_qor_report.rpt
================================================================
  Generated by:           Genus(TM) Synthesis Solution 21.14-s082_1
  Generated on:           Oct 26 2026  04:38:50 pm
  Module:                 harris_top_IMG_W64_IMG_H64
  Operating conditions:   slow (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
================================================================

Timing
--------

Clock Period
-------------
clk   2250.0


  Cost    Critical               Violating
 Group   Path Slack     TNS        Paths
-------------------------------------------
clk         -1465.0   -254620.5        378
default    No paths         0.0
-------------------------------------------
Total                 -254620.5        378

Instance Count
--------------
Leaf Instance Count             71778
Physical Instance count             0
Sequential Instance Count       16741
Combinational Instance Count    55037
Hierarchical Instance Count         9

Area
----
Cell Area                         593021.310
Physical Cell Area                0.000
```

Figure 8: *Genus QoR report: timing summary, instance counts and area*

Power

Table 11: *Genus power report (W converted to mW)*

Group	Leakage (mW)	Internal (mW)	Switching (mW)	Total (mW)	Share
Register	2.356	126.559	2.794	131.710	66.27 %
Logic	1.230	42.433	23.366	67.028	33.73 %
Total	**3.586**	**168.992**	**26.160**	**198.738**	**100 %**
Share	1.80 %	85.03 %	13.16 %	100 %	

Total power is about 199 mW at the 2.25 ns synthesis clock with Genus's default switching activity (no simulation activity was annotated), so it is an estimate for comparison, not a sign-off figure. Two thirds is in the registers and 85 % is internal power: all 16,741 flip-flops are clocked every cycle, and there is no clock gating. At the

final 4 ns clock the dynamic part would scale down roughly in proportion to frequency.

```
[24bvd0069@cadence-server reports_slow]$ cat harris_top_power_report.rpt
Instance: /harris_top_IMG_W64_IMG_H64
Power Unit: W
PDB Frames: /stim#0/frame#0
  ------------------------------------------------------------------------------
    Category         Leakage     Internal    Switching        Total    Row%
  ------------------------------------------------------------------------------
      memory     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
    register     2.35634e-03  1.26559e-01  2.79430e-03  1.31710e-01  66.27%
       latch     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
       logic     1.22954e-03  4.24328e-02  2.33658e-02  6.70282e-02  33.73%
        bbox     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
       clock     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
         pad     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
          pm     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
  ------------------------------------------------------------------------------
    Subtotal     3.58588e-03  1.68992e-01  2.61601e-02  1.98738e-01 100.00%
  Percentage           1.80%       85.03%       13.16%      100.00% 100.00%
  ------------------------------------------------------------------------------
[24bvd0069@cadence-server reports_slow]$
```

Figure 9: *Genus power report*

4.4 Logic Equivalence Checking (Conformal LEC)

Conformal LEC compares the RTL (golden) with the Genus netlist (revised) to prove that synthesis did not change the function. The script (LEC.tcl) reads the slow library, elaborates the RTL with root harris_top and the netlist with root harris_top_IMG_W64_IMG_H64, and then runs the comparison. Three settings matter for this design:

- set_flatten_model -seq_constant treats flip-flops that synthesis removed as constant or unloaded as constants. These 519 golden flip-flops are reported as unreachable rather than as mismatches.
- analyze_datapath -merge is run before compare because harris_response and gradient_products contain several large multipliers, which otherwise tend to end as aborted compare points.
- set_analyze_option -auto lets Conformal automatically re-analyse any point that still aborts.

The first comparison proved 16,768 points equivalent but left **39 flip-flop points aborted**. The automatic abort analysis then resolved all 39 (Figure 10). The final result is:

Table 12: *Conformal LEC result (RTL vs. slow netlist)*

Result	Count
Primary inputs mapped	86 / 86
Primary outputs equivalent	66 / 66
Flip-flop (state) points equivalent	16,741 / 16,741
Total equivalent compare points	16,807

Result	Count
Non-equivalent	0
Aborted	0
Golden flip-flops unreachable (removed by synthesis)	519
Merged state points (sequential merge)	181
Verification report	PASS

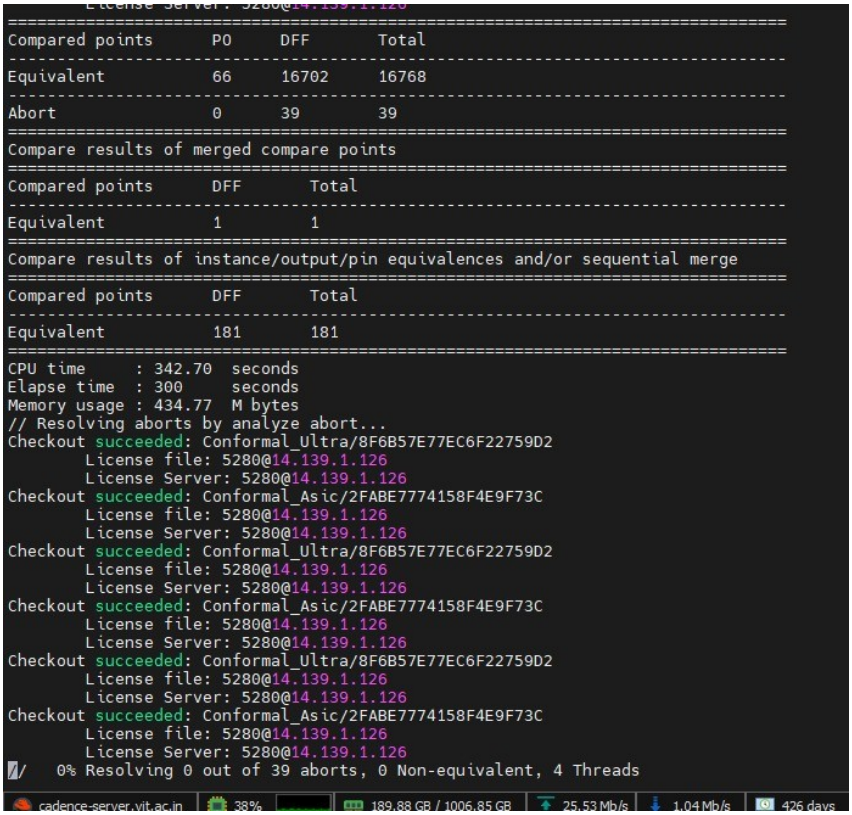


Figure 10: *First comparison: 39 aborted points, followed by automatic abort analysis*

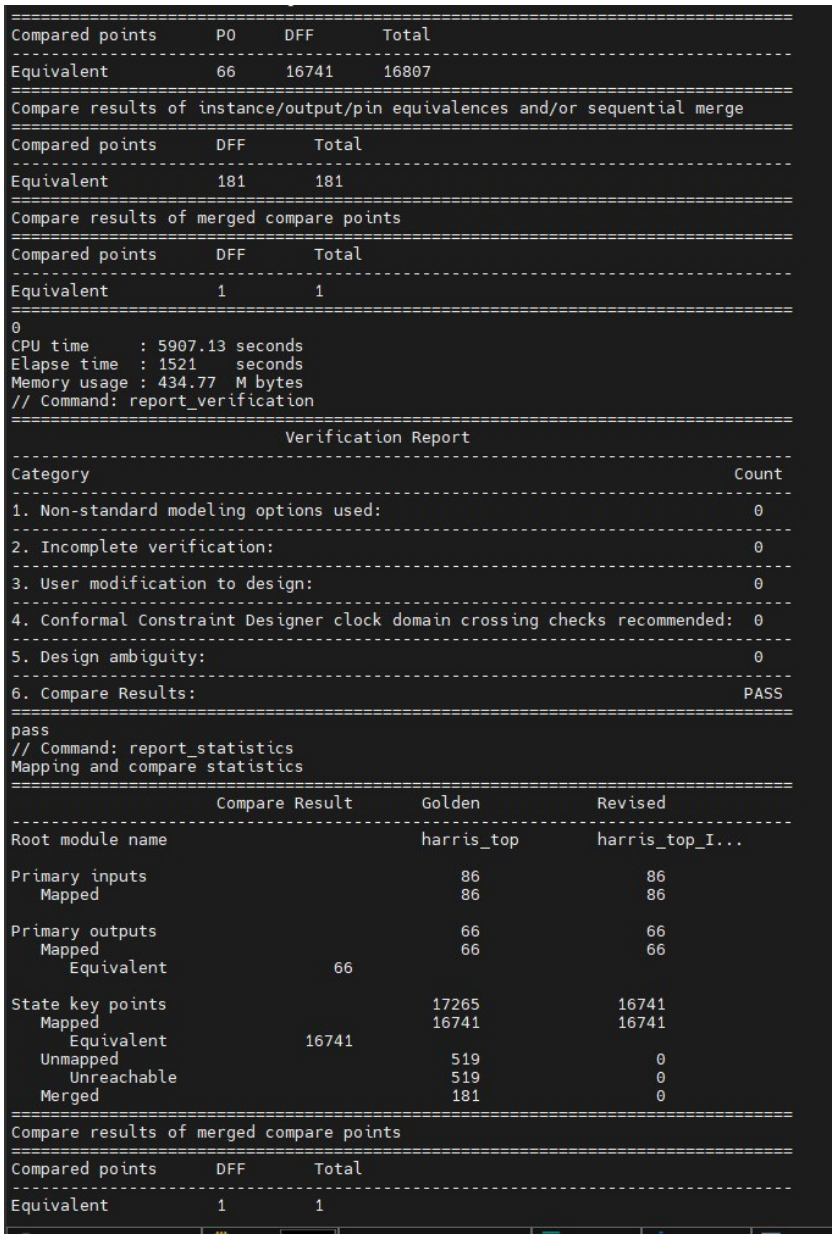


Figure 11: *Final LEC result: verification report PASS and mapping statistics*

4.5 Pre-Layout Static Timing Analysis (Tempus)

Tempus 22.11 reads the slow library, the synthesized netlist and the constraint file, checks the library and the constraints (check_timing), runs a full timing update and reports the worst setup (late) and hold (early) paths. The clock is still ideal at this point, so hold results are only indicative; hold is analysed properly at the fast corner after clock-tree synthesis in the back end.

Table 13: *Tempus pre-layout timing (slow library)*

Check	Worst path	Slack	Status
Setup (late)	u_box/u_xy/sum_out_reg[1] → u_resp/p_sxy2_reg[46]: arrival 3.607 ns, setup 0.144 ns, required 3.656 ns	+0.049 ns	MET
Hold (early)	rst_n → u_win/win_valid_reg (synchronous reset): arrival 0.129 ns, hold 0.075 ns	+0.054 ns	MET

This Tempus run used a 3.8 ns clock (the report shows a phase shift of 3.800 ns), slightly tighter than the final 4 ns. Both checks are met, and the worst setup path is the same Sxy^2 multiplier path found by Genus. With a single clock and an ideal clock tree, moving to 4 ns adds 0.2 ns to every setup slack, so the worst setup slack at 4 ns is at least +0.249 ns before layout.

```
[24bvd0069@cadence-server reports_slow]$ cat setup_tempus.rpt
#############################################################
#  Generated by:      Cadence Tempus 22.11-s001_1
#  OS:                Linux x86_64(Host ID cadence-server.vit.ac.in)
#  Generated on:      Thu Oct 29 00:06:45 2026
#  Design:            harris_top_IMG_W64_IMG_H64
#  Command:           report_timing -late > ./reports_slow/setup_tempus.rpt
#############################################################
Path 1: MET Setup Check with Pin u_resp/p_sxy2_reg[46]/CK
Endpoint:   u_resp/p_sxy2_reg[46]/D     (^) checked with  leading edge of 'clk'
Beginpoint: u_box/u_xy/sum_out_reg[1]/Q (v) triggered by  leading edge of 'clk'
Path Groups: {clk}
Other End Arrival Time          0.000
- Setup                         0.144
+ Phase Shift                   3.800
= Required Time                 3.656
- Arrival Time                  3.607
= Slack Time                    0.049
     Clock Rise Edge                     0.000
     + Clock Network Latency (Ideal)     0.000
     = Beginpoint Arrival Time           0.000
     ------------------------------------------------------------------------------------------
     Instance                                Arc           Cell       Delay  Arrival  Required
                                                                              Time     Time
     ------------------------------------------------------------------------------------------
     u_box/u_xy/sum_out_reg[1]               CK ^          -          -      0.000    0.049
     u_box/u_xy/sum_out_reg[1]               CK ^ -> Q v   SDFFQX4    0.369  0.369    0.418
     u_resp/g51129                           A v -> Y ^    NOR2X1     0.093  0.462    0.511
     u_resp/g51127                           B ^ -> Y ^    CLKAND2X4  0.159  0.621    0.669
     u_resp/g6979                            B ^ -> Y ^    CLKAND2X2  0.142  0.762    0.811
     u_resp/g46522                           A ^ -> Y v    CLKINVX2   0.039  0.802    0.851
     u_resp/g48332                           B v -> Y ^    NAND2X2    0.063  0.865    0.913
     u_resp/g66187                           A ^ -> Y v    NAND2X4    0.082  0.947    0.995
     u_resp/g66557                           A v -> Y ^    INVX2      0.095  1.042    1.091
     u_resp/csa_tree_SUB_TC_OP_groupi_g35806 A ^ -> Y v    CLKINVX2   0.043  1.084    1.133
     u_resp/g65968                           B v -> Y ^    NOR2BX1    0.074  1.159    1.208
     u_resp/g77949                           A1N ^ -> Y ^  AOI2BB1X1  0.155  1.314    1.363
     u_resp/g65849                           B ^ -> Y v    NOR2X2     0.065  1.379    1.428
```

Figure 12: *Tempus setup report: worst path (start)*

```
u_resp/g51129                          A v -> Y ^    NOR2X1     0.093  0.462    0.511
u_resp/g51127                          B ^ -> Y ^    CLKAND2X4  0.159  0.621    0.669
u_resp/g6979                           B ^ -> Y ^    CLKAND2X2  0.142  0.762    0.811
u_resp/g46522                          A ^ -> Y v    CLKINVX2   0.039  0.802    0.851
u_resp/g48332                          B v -> Y ^    NAND2X2    0.063  0.865    0.913
u_resp/g66187                          A ^ -> Y v    NAND2X4    0.082  0.947    0.995
u_resp/g66557                          A v -> Y ^    INVX2      0.095  1.042    1.091
u_resp/csa_tree_SUB_TC_OP_groupi_g35806 A ^ -> Y v   CLKINVX2   0.043  1.084    1.133
u_resp/g65968                          B v -> Y ^    NOR2BX1    0.074  1.159    1.208
u_resp/g77949                          A1N ^ -> Y ^  AOI2BB1X1  0.155  1.314    1.363
u_resp/g65849                          B ^ -> Y v    NOR2X2     0.065  1.379    1.428
u_resp/g470                            A v -> Y v    BUFX3      0.129  1.508    1.557
u_resp/g65641                          A0 v -> Y ^   AOI21X2    0.102  1.610    1.658
u_resp/g122                            B ^ -> Y v    NAND2X1    0.120  1.730    1.779
u_resp/g66609                          B v -> Y ^    NAND2X1    0.068  1.798    1.847
u_resp/g49691                          A ^ -> Y v    CLKINVX1   0.053  1.851    1.900
u_resp/g65618                          A1 v -> Y ^   OAI21X2    0.084  1.935    1.984
u_resp/fopt65439                       A ^ -> Y v    INVX1      0.061  1.996    2.045
u_resp/g66795                          A v -> Y ^    NAND2X2    0.053  2.049    2.098
u_resp/g66794                          B0 ^ -> Y v   OAI21X2    0.080  2.129    2.178
u_resp/g66796                          B v -> Y ^    NAND2X1    0.076  2.205    2.254
u_resp/g44924                          A ^ -> Y v    NAND2X2    0.092  2.296    2.345
u_resp/g49786                          B v -> Y ^    NAND2X2    0.065  2.361    2.410
u_resp/g49785                          B ^ -> Y v    NAND2X2    0.071  2.432    2.481
u_resp/g51421                          B v -> Y ^    XOR3X1     0.289  2.722    2.770
u_resp/g63958                          B ^ -> Y v    NAND2X2    0.083  2.805    2.854
u_resp/g66756                          B v -> Y ^    NAND2X2    0.077  2.882    2.931
u_resp/g66754                          A2 ^ -> Y v   AOI31X4    0.108  2.990    3.039
u_resp/g70355                          A1 v -> Y ^   OAI21X2    0.087  3.077    3.126
u_resp/g64522                          B ^ -> Y v    NAND2X1    0.091  3.168    3.216
u_resp/g65369                          B v -> Y ^    NAND2X2    0.074  3.242    3.290
u_resp/fopt48846                       A ^ -> Y v    CLKINVX2   0.056  3.298    3.347
u_resp/fopt48845                       A v -> Y ^    CLKINVX2   0.048  3.346    3.395
u_resp/g66275                          A1 ^ -> Y v   AOI21X2    0.078  3.424    3.473
u_resp/g66277                          B v -> Y ^    NAND2X1    0.058  3.482    3.531
u_resp/g73634                          B0 ^ -> Y v   OAI21X1    0.062  3.545    3.594
u_resp/g73631                          C v -> Y ^    NAND3XL    0.063  3.607    3.656
u_resp/p_sxy2_reg[46]                  D ^           DFFRHQX8   0.000  3.607    3.656
-------------------------------------------------------------------------------------
```

Figure 13: *Tempus setup report: worst path (continued)*

```
[24bvd0069@cadence-server reports_slow]$ cat hold_tempus.rpt
#============================================================
#  Generated by:      Cadence Tempus 22.11-s001_1
#  OS:                Linux x86_64(Host ID cadence-server.vit.ac.in)
#  Generated on:      Thu Oct 29 00:06:45 2026
#  Design:            harris_top_IMG_W64_IMG_H64
#  Command:           report_timing -early > ./reports_slow/hold_tempus.rpt
#============================================================
Path 1: MET Hold Check with Pin u_win/win_valid_reg/CK
Endpoint:   u_win/win_valid_reg/D (v) checked with  leading edge of 'clk'
Beginpoint: rst_n                 (v) triggered by  leading edge of 'clk'
Path Groups: {clk}
Other End Arrival Time          0.000
+ Hold                          0.075
+ Phase Shift                   0.000
= Required Time                 0.075
  Arrival Time                  0.129
  Slack Time                    0.054
     Clock Rise Edge                     0.000
     + Input Delay                       0.060
     = Beginpoint Arrival Time           0.060
      ------------------------------------------------------------------------
      Instance             Arc          Cell      Delay  Arrival  Required
                                                         Time     Time
      ------------------------------------------------------------------------
      -                    rst_n v      -         -      0.060    0.006
      u_win/g41100         C0 v -> Y ^  OAI211X1  0.033  0.093    0.038
      u_win/g41099         B0 ^ -> Y v  AOI21XL   0.036  0.129    0.075
      u_win/win_valid_reg  D v          DFFQX1    0.000  0.129    0.075
      ------------------------------------------------------------------------

[24bvd0069@cadence-server reports_slow]$ cat setup_tempus.rpt
```

Figure 14: *Tempus hold report: worst path*

5. Discussion of Results

5.1 Interpretation

- **Functionality.** All 28 directed and randomised tests passed in Xcelium: 1,260 outputs checked and 672 corners with no mismatches, including stalls, idle periods, back-to-back frames, mid-frame reset and extreme k_q and thr values. The RTL meets its functional requirements.
- **Equivalence.** Conformal LEC proves all 16,807 compare points equivalent with no non-equivalent or aborted points, so the netlist is functionally identical to the RTL.
- **Timing.** At 2.25 ns the design fails by 1.465 ns because of the Sxy^2 multiplier. With the relaxed clock Tempus reports +0.049 ns setup slack at 3.8 ns (at least +0.249 ns at 4 ns) and +0.054 ns hold slack, so the netlist meets timing before layout.
- **Area and power.** 0.593 mm^2 and about 199 mW (estimate). The 16,741 flip-flops, almost all of them line-buffer storage, take 58.6 % of the area and 66 % of the power.
- **Throughput.** At one pixel per clock and 4 ns, the core processes 250 Mpixel/s, about 61,000 frames/s for a 64 × 64 image.

5.2 Issues Encountered and Resolutions

Table 14: *Issues encountered during the front end*

Issue	Cause	Resolution
Setup violated at 2.25 ns: WNS −1.465 ns, TNS −254.6 ns, 378 paths	26 × 26 Sxy^2 multiplier in one stage of harris_response (3.57 ns data path)	Clock relaxed to 4 ns in the SDC used for STA and implementation
Very tight design-rule limits	max_transition 0.15 ns and max_fanout 4 on high-fanout nets	Relaxed to 0.5 ns and 32
39 aborted points in LEC	Large multipliers in harris_response and gradient_products	analyze_datapath -merge plus automatic abort analysis; 0 aborts in the final run
519 golden flip-flops without a netlist match	Constant or unloaded register bits optimised away by Genus	set_flatten_model -seq_constant; reported as unreachable, not mismatches
Top level renamed by Genus	Elaborating with IMG_W/IMG_H = 64 gives harris_top_IMG_W64_IMG_H64	Script reads the elaborated name; LEC, Tempus and Innovus use the renamed top cell
Loss of hierarchy	Default auto-ungrouping flattens modules	auto_ungroup none keeps per-module area and instance names

5.3 Possible Improvements

- Pipelining the three multipliers in harris_response over two stages would shorten the critical path and could bring the clock back towards the original 2.25 ns target.
- Replacing the flip-flop line buffers with SRAM macros, or applying clock gating to them, would cut most of the area and the register power.